# Trabalho Prático
# Sistema de Análise de Logs

Sistemas Operativos

Pedro Sobral (pmsobral@ufp.edu.pt)

Março 2026

## 1 Objectivo

Desenvolver um **sistema de processamento paralelo de logs em grande escala**, recorrendo aos conhecimentos adquiridos na unidade curricular de Sistemas Operativos. O sistema deve ser capaz de analisar milhões de linhas de logs de forma eficiente, utilizando programação concorrente com processos e threads.

## 2 Definição do Problema

### 2.1 Contexto

Empresas modernas geram milhões de linhas de logs diariamente a partir de:

- Servidores web (Apache, Nginx)
- Aplicações distribuídas (microserviços)
- Sistemas de segurança (firewalls, IDS/IPS)
- Infraestrutura cloud

A análise destes dados em tempo real é crítica para:

- **Deteção de intrusões** e anomalias de segurança
- **Monitorização de performance** e debugging
- **Análise de comportamento** de utilizadores
- **Compliance** e auditoria

### 2.2 Problema a Resolver

Processar **datasets massivos de logs** (centenas de MB a GB) de forma eficiente, extraindo métricas relevantes, detetando padrões suspeitos e gerando relatórios agregados.

O desafio consiste em:

1. **Parsing eficiente**: Interpretar múltiplos formatos de logs (Apache, JSON, Syslog, Nginx)

2. **Processamento paralelo**: Utilizar múltiplos processos/threads para maximizar throughput
3. **Comunicação entre processos**: Implementar mecanismos IPC (pipes, sockets, shared memory)
4. **Sincronização**: Coordenar acesso a recursos partilhados sem race conditions
5. **Agregação de resultados**: Consolidar dados de múltiplos workers

### 2.3 Arquitetura Proposta

```
┌──────────────────────────────────────────┐
│          CONTROLADOR CENTRAL             │
│  - Coordenação de workers                │
│  - Agregação de resultados               │
│  - Dashboard de progresso                │
└──────────────────────────────────────────┘
                │
     ┌──────────┴──────────┐
     ▼                     ▼
┌─────────┐           ┌─────────┐
│ FASE 1  │           │ FASE 2  │
│PROCESSOS│           │ THREADS │
└─────────┘           └─────────┘
```

## 3 Requisitos - Fase 1 (Processos)

### 3.1 A. (10%) Interface de Linha de Comandos

Adaptar o código para receber argumentos de linha de comandos:

```
./logAnalyzer <diretorio_logs> <num_processos> <modo> [opcoes]
```

**Exemplo de utilização:**

```
./logAnalyzer /var/log/apache2 4 security --verbose --output=report.json
```

**Parâmetros obrigatórios:**

- `<diretorio_logs>`: Pasta contendo ficheiros .log a processar
- `<num_processos>`: Número de processos worker a criar
- `<modo>`: Modo de análise
  - `security`: Foco em eventos de segurança
  - `performance`: Análise de tempos de resposta
  - `traffic`: Análise de tráfego e padrões de acesso
  - `full`: Análise completa

**Opções facultativas:**

- `--verbose`: Modo verboso (logs detalhados)
- `--output=<ficheiro>`: Ficheiro de saída do relatório

## 3.2 B. (15%) Arquitetura Multi-Processo Básica

Criar **N processos filho**, cada um responsável por processar um subconjunto dos ficheiros de log encontrados no diretório.

### 3.2.1 Funcionalidades:

**Processo PAI:**

1. Descobrir todos os ficheiros `.log` no diretório especificado
2. Dividir lista de ficheiros entre N processos filho
3. Aguardar terminação de todos os filhos usando `wait()`/`waitpid()`

**Processos FILHO:**

1. Receber subset de ficheiros a processar
2. Para cada ficheiro:
    - Abrir com `open()`
    - Ler linha a linha com `read()`
    - Parsear cada linha conforme formato detetado
    - Extrair métricas:
        - Total de linhas processadas
        - Eventos por tipo (INFO, WARN, ERROR, CRITICAL)
        - Top 10 IPs mais frequentes
        - Erros HTTP (4xx, 5xx) se aplicável
3. Escrever resultados em **ficheiro partilhado** `results_<pid>.txt`

### 3.2.2 Formato de saída por processo:

```
PID:1234;FICHEIRO:access.log;LINHAS:50000;ERRORS:234;WARNINGS:1205
```

**Nota:** Neste requisito **não** existe comunicação entre processos durante execução. Cada processo trabalha de forma independente e apenas escreve resultados no final.

## 3.3 C. (30%) Comunicação via Pipes Anónimos

Alterar o requisito B para que os processos filho **comuniquem via pipes** com o processo pai, em vez de escrever em ficheiros partilhados.

### 3.3.1 Alterações necessárias:

**Processo PAI:**

1. Criar **um** pipe em que todos os filhos escrevem e o pai lê.
2. Após `fork()`, fechar extremidade de escrita no pai e leitura nos filhos
3. Ler resultados de todos os filhos via `readn()`
4. Calcular estatísticas globais agregando dados de todos os workers
5. Imprimir relatório final no terminal

**Processos FILHO:**

1. Fechar extremidade de leitura do pipe
2. Processar ficheiros conforme requisito B
3. Enviar resultados via pipe usando `writen()`

### 3.3.2 Implementação obrigatória de funções auxiliares:

```
ssize_t readn(int fd, void *ptr, size_t n);
ssize_t writen(int fd, const void *ptr, size_t n);
```

Estas funções garantem leitura/escrita completa mesmo em caso de interrupções (EINTR).

### 3.3.3 Modos de operação:

**1. Modo Normal:**

Filhos enviam apenas dados agregados finais:

```
PID:1234;LINHAS:50000;ERRORS:234;WARNINGS:1205;TOP_IP:192.168.1.100
```

**2. Modo Verboso (`--verbose`):**

Filhos enviam cada evento crítico imediatamente:

```
PID:1234;TIMESTAMP:2024-02-15T10:23:45;TYPE:CRITICAL;MSG:Database connect
ion failed;IP:10.0.1.50
```

Isto permite ao processo pai monitorizar eventos em tempo real.

## 3.4 D. (15%) Dashboard de Progresso em Tempo Real

O processo pai deve manter e atualizar periodicamente um **indicador visual de progresso** no terminal.

### 3.4.1 Funcionalidades:

**Estrutura de dados partilhada:**

```
struct WorkerStatus {
    pid_t pid;
    long lines_processed;
    long total_lines;
    float progress_pct;
```

```
    enum {IDLE, WORKING, DONE} state;
};
```

**Mecanismo de atualização:**

- Processos filho enviam updates de progresso usando **sinais (SIGUSR1)** ou via pipe dedicado
- Processo pai atualiza dashboard a cada 1 segundo
- Interface visual no terminal (usar ANSI escape codes)

### 3.4.2 Exemplo de visualização:

```
╔═══════════════════════════════════════════╗
  LOG ANALYZER - Real-time Monitor
╠═══════════════════════════════════════════╣
 Worker 1 [▓▓▓
```

# 4 Requisitos - Fase 2 (Threads)

## 4.1 (20%) Modelo Worker Threads Básico

A tarefa principal (**main thread**) deve criar **W worker threads**, cada uma responsável por processar um subconjunto de linhas de log.

### Funcionalidades:

**Main thread:**

1. Ler todos os ficheiros e dividir em blocos
2. Criar W worker threads com `pthread_create()`
3. Aguardar conclusão com `pthread_join()`
4. Imprimir relatório final

**Worker threads:**

1. Receber ponteiro para o bloco de dados (offset inicio/fim)
2. Processar linhas do bloco
3. Actualizar estruturas partilhadas usando **mutexes (`pthread_mutex_t`)**
4. Retornar ao terminar

## 4.2 (20%) Dashboard de Progresso Multi-Thread

Semelhante ao requisito D da Fase 1, mas com um thread dedicado a este propósito.

## 4.3 (60%) Modelo Produtor-Consumidor com Bounded Buffer

Implementar arquitetura avançada com **P threads produtoras** e **C threads consumidoras**, comunicando via fila partilhada (bounded buffer).

**1. Threads Produtoras (P threads):**

1. Abrir ficheiro de log
2. Ler linha a linha
3. Fazer o *parse* da linha → struct `LogEntry`
4. **Inserir na fila** (bloquear se cheia):

**2. Threads Consumidoras (C threads):**

1. **Remover entrada da fila** (bloquear se vazia):
2. Classificar evento (*security, error, traffic*)
3. Testar padrões de interesse:
    - Tentativas de ataques *brute-force* (5+ falhas/minuto do mesmo IP)
    - Erros HTTP 5xx consecutivos

    - Tráfego anómalo (picos súbitos)
4. Actualizar estatísticas partilhadas (com *mutexes*)

# 5. Formatos de Logs Suportados

O sistema deve suportar parsing de 4 formatos standard de logs:

## 5.1 Apache Combined Log Format

Formato padrão de access logs do Apache e Nginx:

```
192.168.1.100 - - [13/Feb/2024:10:23:45 +0000] "GET /api/users HTTP/1.1"
200 1234 "https://example.com" "Mozilla/5.0"
```

**Campos a extrair:**

- IP do cliente
- Timestamp
- Método HTTP (GET, POST, etc.)
- URL requisitado
- Status code (200, 404, 500, etc.)
- Tamanho da resposta (bytes)
- Referer
- User agent

## 5.2 JSON Structured Logs

Logs estruturados em formato JSON (típico de microserviços):

```
{
  "timestamp": "2024-02-13T10:23:45Z",
  "level": "ERROR",
  "service": "auth-api",
  "message": "Database connection timeout",
  "metadata": {
    "ip": "10.0.1.50",
    "user_id": 12345
  }
}
```

**Campos a extrair:**

- Timestamp (ISO 8601)
- Log level (DEBUG, INFO, WARN, ERROR, CRITICAL)
- Service name
- Message

- Metadata (IP, user_id, etc.)

## 5.3 Syslog Format (RFC 3164)

Formato standard de logs de sistema Unix/Linux:

```
<38>Feb 13 10:23:45 web-server sshd[12345]: Failed password for root from
 192.168.1.100 port 22 ssh2
```

**Campos a extrair:**

- Priority (facility × 8 + severity)
- Timestamp
- Hostname
- Service/daemon
- PID
- Message

**Análise de segurança:**

- Detetar falhas de autenticação
- Detetar tentativas sudo
- Detetar bloqueios de firewall

## 5.4 Nginx Error Log

Logs de erro do Nginx:

```
2024/02/13 10:23:45 [error] 12345#0: *1234 connect() failed (111: Connect
ion refused) while connecting to upstream, client: 192.168.1.100, server:
 example.com
```

**Campos a extrair:**

- Timestamp
- Error level (debug, info, notice, warn, error, crit, alert, emerg)
- PID e TID
- Connection ID
- Error message
- Client IP
- Server name

# 6. Datasets Fornecidos

Serão fornecidos datasets sintéticos realistas para desenvolvimento e testes:

```
datasets/
├── apache/
│   ├── access_100k.log      (~10 MB)
│   ├── access_1M.log        (~100 MB)
│   └── access_10M.log       (~1 GB)
├── json_logs/               (500 ficheiros)
│   ├── app_001.json
│   ├── app_002.json
│   └── ...
├── syslog/
│   └── security_5M.log      (~200 MB)
└── nginx/
    └── error_1M.log         (~50 MB)
```

**Total:** ~1.5 GB

Os alunos podem gerar datasets de qualquer tamanho usando os geradores fornecidos.

# 7. Exemplos de Análises

## 7.1 Modo Security

Detectar eventos de segurança:

```
=== SECURITY REPORT ===

⚠️  ALERTS:
  - Brute Force Detected: IP 192.168.1.100
    (45 failed logins in 2 minutes)

  - SQL Injection Attempt:
    /api/search?q=' OR '1'='1

  - Suspicious User-Agent:
    sqlmap/1.0

📊 STATISTICS:
  - Total Failed Logins: 234
  - Unique Attacker IPs: 12
  - Malformed Requests: 56
  - Firewall Blocks: 89
```

## 7.2 Modo Performance

Analisar tempos de resposta:

```
=== PERFORMANCE REPORT ===

📈 RESPONSE TIMES:
  - P50 (Median):  45ms
  - P95:          230ms
  - P99:         1200ms
  - P99.9:       5600ms

🐌 SLOWEST ENDPOINTS:
  1. /api/reports/generate
     - Avg: 3.4s
     - Count: 34 requests

  2. /api/search
     - Avg: 890ms
     - Count: 1203 requests

📊 STATUS CODE DISTRIBUTION:
  - 2xx: 95.2%
  - 4xx:  3.1%
  - 5xx:  1.7%
```

## 7.3 Modo Traffic

Analisar padrões de tráfego:

```
=== TRAFFIC REPORT ===

👥 TOP 10 IPs:
  1. 192.168.1.100  (12,450 requests)
  2. 10.0.0.50      ( 8,234 requests)
  3. 203.0.113.45   ( 5,677 requests)
  ...

🔗 TOP 10 URLs:
  1. /api/users     (45,231 requests)
  2. /api/products  (32,104 requests)
  3. /             (28,567 requests)
  ...

📊 TRAFFIC PATTERN:
  - Peak hour: 14:00-15:00 (15,234 req/h)
  - Low hour:  03:00-04:00 (   892 req/h)
  - Avg rate: 4,567 requests/hour

🌍 USER AGENTS:
  - Chrome:  62.3%
  - Firefox: 21.4%
```

```
  - Safari:  10.2%
  - Bots:     6.1%
```

# 8. Métricas de Avaliação

## 8.1 Performance (Benchmarks)

Os alunos devem medir e reportar:

**Speedup:**

$$\text{Speedup} = \frac{T_{\text{sequencial}}}{T_{\text{paralelo}}}$$

**Throughput:**

$$\text{Throughput} = \frac{\text{Linhas processadas}}{\text{Tempo (segundos)}}$$

**Eficiência:**

$$\text{Eficiência} = \frac{\text{Speedup}}{N_{\text{cores}}} \times 100\%$$

### Resultados esperados:

| Dataset | 1 Core | 4 Cores | Speedup | Eficiência |
|---|---|---|---|---|
| Apache 100K | 2.0s | 0.6s | 3.3x | 83% |
| JSON 50 files | 1.0s | 0.3s | 3.3x | 83% |
| Syslog 50K | 1.5s | 0.5s | 3.0x | 75% |

## 8.2 Verificação

Verificar a fiabilidade dos resultados:

1. **Comparação com ferramenta de referência:**

```
# Usar awk/grep para validar
awk '$9 >= 400' access.log | wc -l  # Contar erros 4xx/5xx
```

2. **Consistência entre execuções:**

```
# Executar 100 vezes, verificar se resultados são idênticos
for i in {1..100}; do
    ./logAnalyzer datasets/apache/ 4 full > result_$i.txt
done

md5sum result_*.txt | cut -d' ' -f1 | sort -u | wc -l
# Deve retornar 1 (todos os hashes iguais)
```

3. **Validação de agregações:**
    - Total de linhas processadas = soma de linhas de todos os ficheiros
    - Contadores de eventos devem ser consistentes

# 9. Notas de Implementação

## 9.1 Chamadas ao Sistema Obrigatórias

**Em todo o projeto**, os alunos devem utilizar **exclusivamente** as chamadas ao sistema POSIX nativas:

- **I/O:** `open()`, `read()`, `write()`, `close()`, `lseek()`
- **Processos:** `fork()`, `exec()`, `wait()`, `waitpid()`, `kill()`
- **Comunicação IPC:**
    - Pipes: `pipe()`, `socketpair()`
    - Sockets: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`
    - Shared memory: `mmap()`, `shm_open()`, `shm_unlink()`
- **Sincronização:**
    - Threads: `pthread_create()`, `pthread_join()`, `pthread_detach()`
    - Mutexes: `pthread_mutex_init()`, `pthread_mutex_lock()`, `pthread_mutex_unlock()`
    - Condition Variables: `pthread_cond_init()`, `pthread_cond_wait()`, `pthread_cond_signal()`
    - Semáforos: `sem_init()`, `sem_wait()`, `sem_post()`

**NÃO utilizar:**

- Funções de biblioteca C standard: `fopen()`, `fread()`, `fwrite()`, `fprintf()`, etc.

## 9.2 Gestão de Erros

**Todas** as chamadas ao sistema devem verificar erros usando `perror()`:

```
int fd = open("file.log", O_RDONLY);
if (fd < 0) {
    perror("open");
    exit(EXIT_FAILURE);
}

ssize_t n = read(fd, buffer, sizeof(buffer));
if (n < 0) {
    perror("read");
    close(fd);
```

```
    exit(EXIT_FAILURE);
}
```

# 10. Entrega e Avaliação

## 10.1 Modalidade

- Trabalho realizado **individualmente** ou em **grupos de 2 alunos**
- Submissão via CANVAS
- Defesa presencial obrigatória ("viva voz")

## 10.2 Componentes de Entrega

**1. Código fonte completo:**

- Todos os ficheiros `.c` e `.h`
- `Makefile` funcional
- Scripts de teste (opcional)

**2. Relatório (formato PDF):**

- Introdução e objectivos
- Descrição da arquitectura implementada
- Decisões de design (justificadas)
- Resultados experimentais:
  - Tabelas de performance (speedup, throughput)
  - Gráficos comparativos
  - Análise de escalabilidade
- Dificuldades encontradas e soluções
- Conclusões
- **Máximo 15 páginas** (excluindo anexos)

**3. README.md:**

```
# Log Analyzer - Sistemas Operativos

## Autores
- Nome Completo (email@ufp.edu.pt)
- Nome Completo (email@ufp.edu.pt)

## Compilação
make

## Execução
./logAnalyzer <dir> <processos> <modo> [opcoes]
```

```
## Testes
make test

## Estrutura
...
```

**4. Ficheiro comprimido:**

```
projeto_SO_<nome1>_<nome2>.tar.gz
ou
projeto_SO_<nome1>_<nome2>.zip
```

## 10.3 Critérios de Avaliação

### Fase 1 (50%)

| Requisito | Peso |
|---|---|
| A - CLI | 10% |
| B - Multi-processo básico | 15% |
| C - Comunicação via pipes | 30% |
| D - Dashboard progresso | 15% |
| E - Unix Domain Sockets | 30% |

### Fase 2 (50%)

| Requisito | Peso |
|---|---|
| A - Worker threads básico | 20% |
| B - Dashboard threads | 20% |
| C - Produtor-consumidor | 60% |

# 11. Recursos e Bibliografia

## 11.1 Livros Recomendados


1. **Advanced Programming in the UNIX® Environment** (3rd Edition)
   W. Richard Stevens, Stephen A. Rago
   Addison-Wesley, 2013

2. **The Linux Programming Interface**
   Michael Kerrisk
   No Starch Press, 2010

3. **Operating Systems: Three Easy Pieces**
   Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau
   Disponível gratuitamente: https://pages.cs.wisc.edu/~remzi/OSTEP/

## 11.2 Recursos Online

- **POSIX Threads Programming:**
  https://computing.llnl.gov/tutorials/pthreads/
- **Beej's Guide to Unix IPC:**
  https://beej.us/guide/bgipc/
- **Apache Log Format Documentation:**
  https://httpd.apache.org/docs/current/logs.html
- **Syslog RFC 3164:**
  https://tools.ietf.org/html/rfc3164

## 11.3 Ferramentas Úteis

- **Valgrind:** Deteção de memory leaks

```
valgrind --leak-check=full ./logAnalyzer ...
```

- **GDB:** Debugging

```
gdb ./logAnalyzer
```

- **strace:** Trace de system calls

```
strace -c ./logAnalyzer ...  # Estatísticas
```

- **perf:** Profiling de performance

```
perf stat ./logAnalyzer ...
```

---